

HiGreen-GTrXL: Hierarchical Reinforcement Learning for Carbon-Aware Geo-Distributed Datacentre Scheduling with Renewable Look-Ahead

1st Mingwei Li
School of Computer Science
University of Exeter
Exeter, United Kingdom
ml969@exeter.ac.uk

2nd Lu Liu
School of Computer Science
University of Exeter
Exeter, United Kingdom
l.liu3@exeter.ac.uk

3rd Mengqi Li
School of Computer Science
University of Exeter
Exeter, United Kingdom
ml1065@exeter.ac.uk

Abstract—Carbon emissions and energy consumption are rapidly rising, and their side effects—including greenhouse gas accumulation and sea-level rise—pose severe risks to human society. The datacentres are critical infrastructure for modern digital services (e.g., GenAI), yet they consume massive amounts of electricity across computing and cooling subsystems, and produce a large amount of carbon. While integrating renewable energy is essential for decarbonisation, renewable generation is intermittent and volatile, and storage remains expensive, causing persistent supply–demand imbalance and substantial green-energy curtailment due to temporal mismatch (e.g., over 25% of wind power is wasted in the UK during local overproduction and mismatch). Existing strategies are still largely reactive, relying on grid reconfiguration or carbon offsetting rather than proactively aligning renewable supply with computing demand.

This work studies bilateral green-energy alignment for multi-data-centre cloud scheduling and proposes *HiGreen-GTrXL*, a hierarchical DRL (deep reinforcement learning) approach with a “God-eye” green-power prediction module. The forecasting module follows a look-back–to–horizon paradigm: using the most recent look-back window of green power, it predicts future renewable availability over a horizon, producing both short-term and long-term renewable energy look-ahead signals for each datacentre. For efficiency and consistency, we pre-compute the multi-horizon forecasts for each simulation step at the beginning of an episode and inject these look-ahead features into the global state during the experiment. *HiGreen-GTrXL* then performs two-level decision making: a global policy routes arriving tasks across geographically distributed datacentres conditioned on the predicted green supply, while local policies schedule tasks to VMs (virtual machines) within each datacentre, jointly optimising carbon-aware objectives while maintaining service quality.

Experimental results show that our hierarchical approach achieves the best performance across key KPIs, yielding the lowest carbon emissions and carbon intensity. Compared with strong baselines (round-robin, GA, PSO variants, and single-agent PPO methods), our method reduces total carbon emissions by 13.6%–16.4% and carbon intensity by 8.2%–15.4% at the 100k cloudlet scale.

Index Terms—Multi-Agent Reinforcement Learning, Green Computing, Carbon-aware Scheduling, Transformer-XL

I. INTRODUCTION

Geo-distributed datacentres underpin modern cloud services and latency-sensitive applications. However, the rapid growth of computational demand has made datacentre electricity consumption and associated carbon emissions a pressing concern [1]. At the same time, renewable generation is increasingly integrated into power systems, yet its intermittency and limited storage lead to substantial curtailment due to supply–demand mismatch. This motivates datacentre operators to improve *temporal and spatial alignment* between computing demand and renewable availability, rather than relying solely on grid-side actions such as transmission and offsetting.

A promising direction is to shift the focus from *moving energy to computation* towards *moving computation to energy*. In practice, this requires jointly deciding (i) where to route arriving tasks across datacentres and (ii) how to dispatch tasks to VMs within each datacentre, while accounting for heterogeneous infrastructure, region-dependent carbon intensity, and time-varying renewable supply. Such a decision process is sequential and partially observable, and must trade off carbon reduction against QoS, e.g., waiting time and queue backlog.

Recent work has shown that DRL can improve efficiency in datacentre scheduling [2]–[4]. Nevertheless, many approaches primarily optimise within a single datacentre, or react to instantaneous conditions without explicitly incorporating look-ahead renewable signals and cross-datacentre coordination. As a result, opportunities to proactively route workload towards locations with forthcoming renewable surplus can be under-exploited.

In this work, *HiGreen-GTrXL*, a hierarchical DRL framework for geo-distributed scheduling with green-power look-ahead, is developed. *HiGreen-GTrXL* decomposes decision making into two levels: a global policy routes tasks across datacentres based on aggregated operational states and renewable look-ahead features, while local policies dispatch tasks to feasible VMs within each datacentre using action masking. We model datacentre power using utilisation-driven host power models and compute carbon emissions using region-dependent

carbon factors, enabling carbon-aware optimisation under heterogeneous infrastructures. Experiments in a CloudSim Plus-based simulator, integrated with PettingZoo and trained via Ray RLlib, demonstrate consistent reductions in brown energy consumption and carbon emissions compared with strong heuristic and RL baselines.

Our main contributions are:

- A geo-distributed workload orchestration problem is formulated as a hierarchical routing-and-placement task that jointly accounts for carbon emissions, renewable curtailment, and QoS.
- *HiGreen-GTrXL* is proposed as a hierarchical DRL scheduler with a global routing policy and feasibility-aware local VM placement via action masking.
- A multi-scale renewable look-ahead representation is constructed (short-term mean/trend and long-term mean/peak-timing) and injected into the global observation, enabling foresight-driven routing under intermittent renewable supply.
- An extensible CloudSim Plus-based evaluation environment is built with region-dependent carbon factors and heterogeneous server power profiles derived from SPECpower_ss2008 [5], and extensive comparisons are conducted across multiple workload intensities and strong baselines.

II. RELATED WORK

This section reviews prior work related to renewable-aware task scheduling in cloud datacentres, with an emphasis on geo-distributed orchestration and carbon-aware objectives. Existing approaches can be broadly categorised into (i) heuristics and rule-based controllers, (ii) mathematical optimisation, (iii) meta-heuristics, and (iv) learning-based (AI/DRL) scheduling. We summarise representative techniques and highlight the gap addressed by our work.

A. Heuristics and Rule-based Scheduling

Early cloud schedulers often relied on static policies such as Round Robin and Shortest Job First due to their simplicity and low overhead, but these methods typically lack adaptivity under dynamic workloads and heterogeneous resources [6]. Extensions incorporating energy-awareness commonly apply hand-crafted rules, for instance DVFS-based heuristics that adjust processor frequency/voltage to reduce power while meeting task constraints [7]. Renewable-aware variants may defer or prioritise tasks based on instantaneous renewable availability or storage state, improving local renewable utilisation but remaining reactive and sensitive to workload surges [8].

Hierarchical control structures have also been explored, combining a coarse-grained global controller with fine-grained local controllers for VM/host-level management [9]. While conceptually aligned with hierarchical scheduling, such rule-based frameworks are often limited by fixed decision logic and do not naturally integrate predictive look-ahead signals or a unified optimisation objective across sites. Moreover,

topology-aware heuristics (e.g., joint server–network scheduling for DAG workflows) can reduce communication and switch energy, but typically require workload structure assumptions and may be difficult to generalise across diverse cloud traces [10].

B. Mathematical Optimisation

Mathematical optimisation models (e.g., MILP/MINLP) provide principled formulations for jointly optimising cost, energy, and QoS constraints in datacentres. For instance, queueing-aware formulations with DVFS-enabled servers can yield near-optimal operating points for hybrid-powered geo-distributed datacentres, but at substantial computational cost that limits real-time applicability at scale [11]. Related work also applies MILP to design and operate hybrid renewable systems for datacentres, achieving deterministic solutions under strong assumptions on forecasts and parameters [12].

Computation shifting has been studied as an alternative to physical storage or transmission, such as the Virtual Battery paradigm that migrates workload to renewable-rich regions; however, frequent migration may induce high network overhead and complicate operational constraints [12]. Optimisation-based energy management in fully renewable or multi-energy systems similarly offers detailed modelling, yet often focuses on day-ahead planning and remains sensitive to uncertainty and modelling complexity [13], [14]. Overall, optimisation methods are strong baselines but frequently face scalability and adaptivity challenges in large, stochastic environments.

C. Meta-heuristics

Meta-heuristic algorithms (e.g., GA/PSO/ACO and hybrids) are widely used for multi-objective scheduling when problems are NP-hard and constraints are complex [15]–[17]. They have been applied to workflow scheduling across geo-distributed datacentres and to energy-aware scheduling in fog/cloud settings, often combining global search (exploration) with local refinement (exploitation) [18]–[22]. While these methods can obtain high-quality solutions, they typically require careful parameter tuning, can be computationally intensive, and do not naturally support online learning from streaming arrivals and non-stationary renewable supply.

D. Learning-based Scheduling and Forecast-aware Control

Learning-based approaches, especially DRL, have gained traction for sequential scheduling under uncertainty [23]. Recent systems integrate prediction models and DRL controllers to improve renewable utilisation and reduce brown energy consumption while maintaining QoS [24], [25]. Other work combines RL with optimisation (e.g., ILP/MPC) or introduces uncertainty-aware forecasting (e.g., conformal prediction) to improve robustness to renewable variability [26].

For geo-distributed settings, prior work has explored RL-based workload shifting to exploit spatial diversity in renewable availability [27]–[30]. DRL-based schedulers such as GreenDRL and RARE demonstrate substantial benefits in

energy/carbon reduction under appropriate reward designs and observability assumptions [2], [3]. Multi-agent reinforcement learning (MARL) further enables decentralised control and can address partial observability via centralised training and decentralised execution (CTDE), e.g., combining global routing decisions with local placement decisions [31].

Across these lines of work, a recurring limitation is that many schedulers are either reactive to instantaneous signals or focus on local optimisation within a single datacentre, under-utilising the opportunity to proactively align workload placement with forthcoming renewable availability across sites. In contrast, our work targets *hierarchical geo-distributed scheduling* with explicit renewable look-ahead signals and carbon-aware accounting. We build upon the DRL paradigm and adopt a two-level routing-and-placement structure, enabling system-level carbon optimisation while preserving QoS through feasibility-aware local scheduling.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Assumptions

HiGreen-GTrXL serves as a workload scheduler for geo-distributed datacentres, routing arriving cloud tasks to virtual machines across datacentres. We assume that each task's resource demand is known upon arrival and that the system can observe each datacentre's operational status (e.g., utilisation and queue length). The datacentres are heterogeneous in their infrastructure configurations (host profiles/power models, capacities, and VM fleets), renewable energy supply (Wind turbines & Solar panels), and use region-dependent carbon factors [32]. Carbon emissions are accounted based on energy consumption from different sources (green vs. grid electricity) using the configured emission factors.

For each datacentre, we assume access to a renewable-power forecaster following a look-back-to-horizon formulation: given a look-back window of length L , it outputs a multi-step forecast over horizon H , $\hat{\mathbf{g}}_{i,t:t+H} = f(\mathbf{g}_{i,t-L:t})$. In our setting, we set $L = 96$ and inject four look-ahead features per datacentre into the global state: short-term mean and trend over the next $H_s = 3$ steps (30 minutes), and long-term mean and peak timing over the next $H_l = 144$ steps (24 hours), assuming a 10-minute resolution. As a representative forecasting accuracy level, *TimeCAP* [33] reports MSE = 0.171 and MAE = 0.269 on the Electricity benchmark.

B. System Model

Fig. 1 illustrates the high-level architecture of our hierarchical scheduling system for geo-distributed datacentres. We consider N datacentres that are heterogeneous in their infrastructure configurations (host profiles, VM fleets, and region-dependent carbon factors). Each datacentre is associated with a renewable-power trace (wind/solar) that is aligned to its geographic region via time-zone offsets. At runtime, each datacentre draws IT power from its host power models; renewable energy is used first, and any remaining demand is supplied by grid electricity, while excess renewable generation is curtailed (no storage).

The system follows a two-level hierarchy. At the *global level*, a global scheduler performs inter-datacentre routing: arriving cloud tasks are appended to a global waiting queue, and the scheduler periodically selects a batch of tasks from this queue and assigns each task to a target datacentre. At the *local level*, each datacentre is managed by a local broker and a local scheduler, which maintains a FIFO waiting queue and dispatches tasks to feasible VMs within the same datacentre.

Both the global scheduler and the local schedulers are instantiated as reinforcement learning agents. During inference, the global agent consumes a datacentre-level aggregated observation (including operational metrics and renewable look-ahead features) and outputs routing actions for the current batch. After routing, tasks are transferred to the corresponding local waiting queues, and each local agent repeatedly selects tasks from its FIFO queue and assigns them to VMs based on host/VM utilisation and resource availability.

C. Energy and Carbon Emission Model

This subsection specifies how datacentre-level energy consumption and carbon emissions are computed in the simulator.

Host utilisation and IT power. Let $u_{i,h}(t) \in [0, 1]$ denote the CPU utilisation of host $h \in \mathcal{H}_i$ in datacentre DC_i at simulation time t . The instantaneous IT power of host h is obtained from its utilisation-driven power model:

$$p_{i,h}(t) = P_{i,h}(u_{i,h}(t)). \quad (1)$$

The datacentre IT power is the sum of all hosts:

$$p_i(t) = \sum_{h \in \mathcal{H}_i} p_{i,h}(t). \quad (2)$$

Step energy accounting. Consider a simulation step with elapsed wall-clock time Δt (hours) between two consecutive updates. The IT energy demand of DC_i during this step is

$$\Delta E_i^{\text{dem}}(t) = p_i(t) \Delta t \quad (\text{Wh}). \quad (3)$$

Renewable supply and green-first allocation (no storage). Let $g_i(t)$ denote the instantaneous renewable power available at DC_i (W), aggregated from renewable traces (wind/solar) at time t . The available renewable energy within the step is

$$\Delta E_i^{\text{avail}}(t) = g_i(t) \Delta t \quad (\text{Wh}). \quad (4)$$

We assume a green-first policy with no storage:

$$\Delta E_i^{\text{green}}(t) = \min(\Delta E_i^{\text{dem}}(t), \Delta E_i^{\text{avail}}(t)), \quad (5)$$

$$\Delta E_i^{\text{brown}}(t) = \Delta E_i^{\text{dem}}(t) - \Delta E_i^{\text{green}}(t), \quad (6)$$

$$\Delta E_i^{\text{waste}}(t) = \Delta E_i^{\text{avail}}(t) - \Delta E_i^{\text{green}}(t). \quad (7)$$

Carbon emissions and carbon intensity. Each datacentre DC_i is associated with region-dependent carbon factors η_i^{brown} and η_i^{green} (kgCO₂/kWh) for grid electricity and renewable electricity, respectively. Carbon emissions for the step are computed as

$$\Delta C_i(t) = \eta_i^{\text{green}} \frac{\Delta E_i^{\text{green}}(t)}{1000} + \eta_i^{\text{brown}} \frac{\Delta E_i^{\text{brown}}(t)}{1000} \quad (\text{kgCO}_2), \quad (8)$$

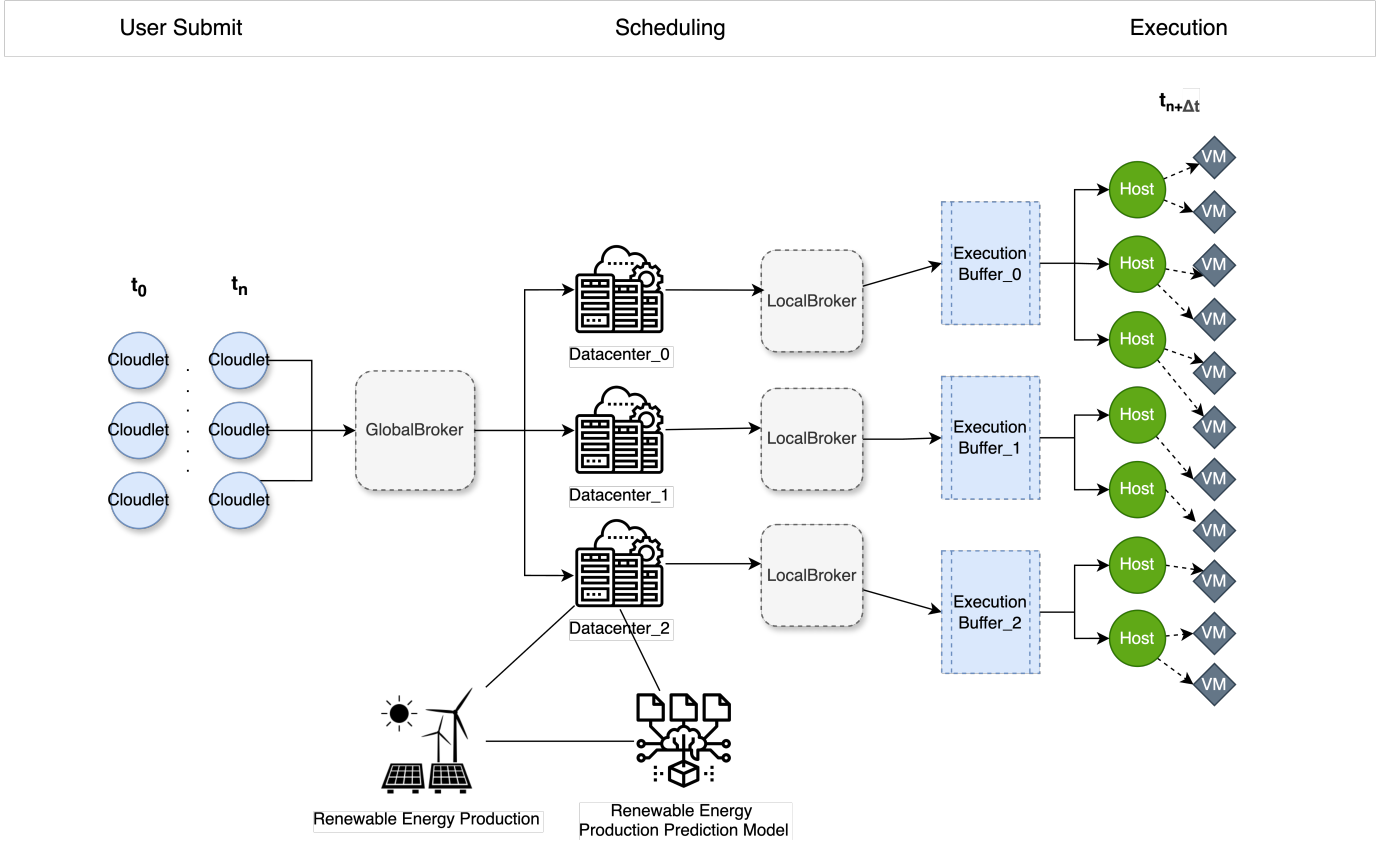


Fig. 1. High-level overview of hierarchical system scheduling architecture in multi-datacentre environments.

and cumulative carbon emissions are $C_i = \sum_t \Delta C_i(t)$. Let $E_i = \sum_t (\Delta E_i^{\text{green}}(t) + \Delta E_i^{\text{brown}}(t))$ be the total consumed energy (Wh) in an episode; the datacentre carbon intensity is

$$\text{CI}_i = \frac{C_i}{E_i/1000} \quad (\text{kgCO}_2/\text{kWh}). \quad (9)$$

D. Problem Formulation

We consider N geo-distributed datacentres. Datacentre i has a fixed (heterogeneous) infrastructure configuration, including a host pool and a VM fleet (Small/Medium/Large types). Cloud tasks (cloudlets) arrive over time; each task c has a resource demand $d(c)$ (e.g., required PEs and workload size in MI) known upon arrival. Each datacentre is associated with a renewable-power trace $g_i(t)$ (wind/solar) aligned to its region (via time-zone offsets) under a compressed time scaling, where one trace row corresponds to a 10-minute renewable interval. Energy and carbon metrics follow Sec. III-C (Eqs. (3)–(9)).

Hierarchical decision making. At each step t , a global routing decision assigns a batch of B tasks from the global waiting queue to datacentres: $a_t^g = (a_{t,1}, \dots, a_{t,B}) \in \{1, \dots, N\}^B$. Within each datacentre i , a local scheduling decision assigns waiting tasks to feasible VMs (or defers execution).

MDP objective. We formulate the problem as a hierarchical Markov decision process (MDP) with a global routing policy π^g and local scheduling policies $\{\pi_i^l\}_{i=1}^N$. Our objective is

to jointly (i) reduce system-wide carbon emissions, (ii) minimise renewable-energy curtailment, and (iii) maintain service quality (QoS). Concretely, we seek a hierarchical policy $\pi = (\pi^g, \{\pi_i^l\}_{i=1}^N)$ that minimises the discounted multi-objective cost:

$$\min_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t \left(\beta \sum_{i=1}^N \Delta C_i(t) + \gamma R_w(t) + \alpha \mathcal{L}_{\text{QoS}}(t) \right) \right], \quad (10)$$

where $\sum_i \Delta C_i(t)$ is the total carbon emitted at step t (Eq. (8)),

$$R_w(t) = \frac{\sum_i \Delta E_i^{\text{waste}}(t)}{\sum_i (\Delta E_i^{\text{waste}}(t) + \Delta E_i^{\text{green}}(t))} \quad (11)$$

is the renewable waste ratio, and

$$\mathcal{L}_{\text{QoS}}(t) = \frac{1}{N} \sum_{i=1}^N \left(\lambda_1 \bar{T}_i^{\text{wait}}(t) + \lambda_2 Q_i(t) \right) \quad (12)$$

penalises the average waiting time $\bar{T}_i^{\text{wait}}(t)$ and queue length $Q_i(t)$ at datacentre i . The global agent mainly optimises carbon emissions and renewable curtailment at the system level, while each local agent focuses on reducing waiting time/queue backlog and ensuring feasible VM assignments within its datacentre.

TABLE I
NOTATION USED IN THE PROPOSED SYSTEM

Notation	Meaning
N	Number of geo-distributed datacentres
DC_i	Datacentre i
$\mathcal{H}_i$	Host set in DC_i
$\mathcal{V}_i$	VM set (fleet) in DC_i
c	A cloud task (cloudlet)
$d(c)$	Resource demand of task c (e.g., required PEs and MI)
$u_{i,h}(t)$	CPU utilisation of host $h \in \mathcal{H}_i$ at time t
$p_{i,h}(t)$	IT power of host $h \in \mathcal{H}_i$ at time t (W)
$p_i(t)$	IT power of DC_i at time t (W)
$g_i(t)$	Available renewable power at DC_i at time t (W)
Δt	Step duration used for energy accounting (hours)
$\Delta E_i^{\text{dem}}(t)$	IT energy demand of DC_i during step t (Wh)
$\Delta E_i^{\text{avail}}(t)$	Available renewable energy of DC_i during step t (Wh)
$\Delta E_i^{\text{green}}(t)$	Green energy used by DC_i during step t (Wh)
$\Delta E_i^{\text{brown}}(t)$	Grid (brown) energy used by DC_i during step t (Wh)
$\Delta E_i^{\text{waste}}(t)$	Wasted renewable energy of DC_i during step t (Wh)
η_i^{brown}	Brown electricity carbon factor (kgCO ₂ /kWh)
η_i^{green}	Green electricity carbon factor (kgCO ₂ /kWh)
$\Delta C_i(t)$	Carbon emitted by DC_i during step t (kgCO ₂)
C_i	Cumulative carbon emissions of DC_i over an episode (kgCO ₂)
E_i	Total energy consumption of DC_i over an episode (Wh)
CI_i	Carbon intensity of DC_i (kgCO ₂ /kWh)
$\mu_i^{\text{short}}(t)$	Short-term renewable mean (God-eye)
$\tau_i^{\text{short}}(t)$	Short-term renewable trend (God-eye)
$\mu_i^{\text{long}}(t)$	Long-term renewable mean (God-eye)
$\phi_i^{\text{peak}}(t)$	Long-term renewable peak timing (God-eye)
$Q_i(t)$	Queue length (waiting tasks) at DC_i
$\bar{T}_i^{\text{wait}}(t)$	Average waiting time at DC_i
$R_w(t)$	Renewable waste ratio at step t
$\mathcal{L}_{\text{QoS}}(t)$	QoS penalty at step t
α, β, κ	Weights for QoS, carbon, and renewable-waste objectives
λ_1, λ_2	Weights for waiting-time and queue-length terms
γ	Discount factor
s_t^g, a_t^g	Global state and routing action at step t
$s_{i,t}^l, a_{i,t}^l$	Local state and VM-assignment action for DC_i at step t
π	Scheduling policy (hierarchical policy $\pi = (\pi^g, \{\pi_i^l\})$)

IV. HiGreen-GTRXL: HIERARCHICAL SCHEDULING WITH GREEN-POWER LOOK-AHEAD

A. Overview and Learning Workflow

This section presents the proposed method, *HiGreen-GTrXL*, a hierarchical DRL scheduler trained with Proximal Policy Optimisation (PPO) and augmented with green-power look-ahead. HiGreen-GTrXL decomposes decision making into two coupled levels: a *global routing policy* π^g that assigns arriving tasks to datacentres, and *local scheduling policies* $\{\pi_i^l\}_{i=1}^N$ that dispatch tasks to feasible VMs within

each datacentre. Both levels are trained using PPO, and are parameterised by a temporal policy backbone (GTrXL) to capture temporal dependencies in workload dynamics and renewable availability. The overall hierarchical routing-and-placement procedure is summarised in Algorithm 2.

A distinguishing feature of HiGreen-GTrXL is the integration of *TimeCAP* (Temporal Carbon-Aware Prediction), a look-ahead mechanism that provides the global agent with multi-horizon forecasts of renewable energy availability. At each decision step, TimeCAP queries future green power trajectories from a prediction module (or ground-truth data in our God-eye setting) and extracts four features per datacentre: short-term mean μ_i^{short} and trend τ_i^{short} over the next k_s intervals (e.g., 30 minutes), and long-term mean μ_i^{long} and peak timing ϕ_i^{peak} over k_l intervals (e.g., 24 hours). These features enable the global policy to anticipate periods of high or low renewable availability and route workloads proactively, rather than reacting solely to current conditions.

At each decision step t , the global agent first invokes TimeCAP to obtain look-ahead features for all datacentres, then collects an aggregated observation s_t^g that combines per-datacentre operational metrics, TimeCAP features, and the current task batch descriptors (Algorithm 1). The global policy π^g outputs a routing action $a_t^g \in \{1, \dots, N\}^B$ that assigns each task in the batch to a target datacentre. Conditioned on this routing, each datacentre i exposes a local observation $s_{i,t}^l$ to its local policy, which then outputs a VM-selection action $a_{i,t}^l$ from a feasibility-constrained action set (implemented via action masking) (Algorithm 2). After executing the global and local actions, the environment advances to the next step and returns the next observations $(s_{t+1}^g, \{s_{i,t+1}^l\})$, together with reward signals reflecting the carbon-waste-QoS trade-off. Transitions collected from the global and local policies are stored in rollout buffers, and PPO updates are performed periodically using the collected trajectories. During training, local policies may share parameters across datacentres to improve sample efficiency, while execution remains decentralised.

B. State and Action

We adopt a two-level hierarchy with a global routing policy π^g and N local scheduling policies $\{\pi_i^l\}_{i=1}^N$. At each decision step t , the global policy routes a batch of B waiting tasks across datacentres, while each local policy dispatches tasks to feasible VMs within its datacentre.

Global state. The global observation s_t^g is a datacentre-level aggregation augmented with green-power look-ahead and batch-level task features:

$$s_t^g = \left(\{\mathbf{g}_i(t)\}_{i=1}^N, \mathbf{c}^{\text{batch}}(t), L_{\text{imb}}(t), C_{\text{recent}}(t) \right). \quad (13)$$

For each datacentre DC_i , the aggregated state vector $\mathbf{g}_i(t)$ is defined as

$$\mathbf{g}_i(t) = \left(P_i^{\text{green}}(t), P_i^{\text{tot}}(t), \rho_i^{\text{green}}(t), W_i^{\text{green}}(t), \mu_i^{\text{short}}(t), \tau_i^{\text{short}}(t), \mu_i^{\text{long}}(t), \phi_i^{\text{peak}}(t), Q_i(t), U_i^{\text{cpu}}(t), A_i^{\text{pes}}(t), U_i^{\text{ram}}(t) \right). \quad (14)$$

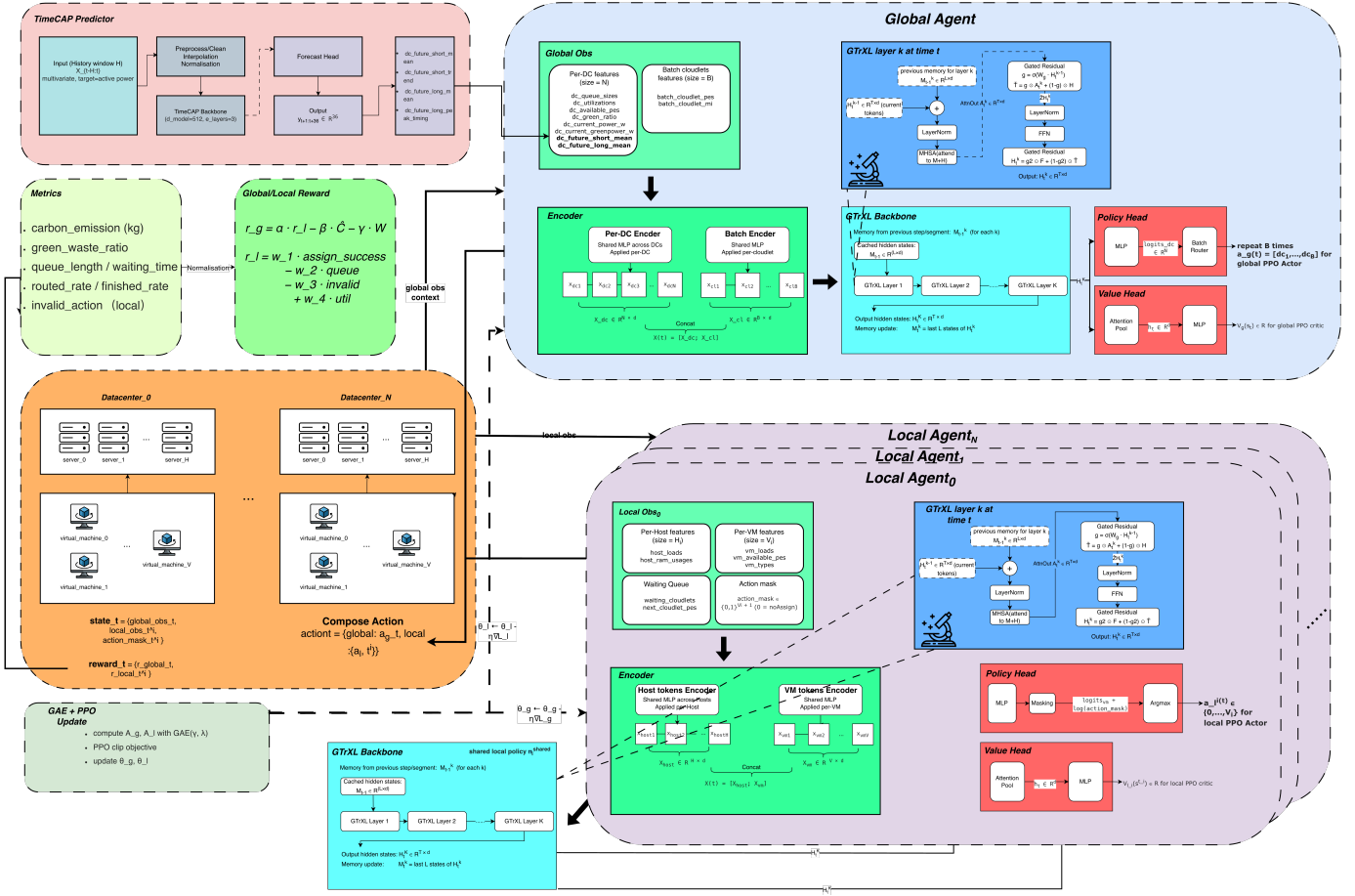


Fig. 2. High-level overview of hierarchical system scheduling architecture in multi-datacentre environments.

where $P_i^{\text{green}}(t)$ is the current available green power (W), $P_i^{\text{tot}}(t)$ is the current IT power draw (W), $\rho_i^{\text{green}}(t) \in [0, 1]$ is the green-energy usage ratio, and $W_i^{\text{green}}(t)$ is the cumulative wasted green energy (Wh). The tuple $(\mu_i^{\text{short}}(t), \tau_i^{\text{short}}(t), \mu_i^{\text{long}}(t), \phi_i^{\text{peak}}(t))$ denotes the God-eye look-ahead features, i.e., short-term mean and trend, and long-term mean and peak timing of renewable availability (Sec. X). Finally, $Q_i(t)$ is the local waiting-queue length, $U_i^{\text{cpu}}(t) \in [0, 1]$ and $U_i^{\text{ram}}(t) \in [0, 1]$ denote average host CPU/RAM utilisation, and $A_i^{\text{pes}}(t)$ is the total number of available PEs across active VMs in DC_i .

The batch-level task features are

$$\mathbf{c}^{\text{batch}}(t) = (C^{\text{up}}(t), \{p_{t,j}^{\text{req}}\}_{j=1}^B, \{m_{t,j}^{\text{req}}\}_{j=1}^B, \mathbf{d}^{\text{pes}}(t)), \quad (15)$$

where $C^{\text{up}}(t)$ is the number of tasks currently waiting in the global queue, $p_{t,j}^{\text{req}}$ and $m_{t,j}^{\text{req}}$ are the PE and MI demand of the j -th task in the routed batch (padded with zeros if fewer than B tasks are available), and $\mathbf{d}^{\text{pes}}(t) \in \mathbb{N}_0^3$ is the PE-demand distribution of waiting tasks (small/medium/large). We further include cross-datacentre statistics such as utilisation imbalance $L_{\text{imb}}(t)$ (e.g., variance across $\{U_i^{\text{cpu}}(t)\}$) and recent completion count $C_{\text{recent}}(t)$.

Global action. The global policy outputs a routing vector

$$\mathbf{a}_t^g = (a_{t,1}^g, \dots, a_{t,B}^g) \in \{1, \dots, N\}^B, \quad (16)$$

where $a_{t,j}^g = k$ assigns the j -th task in the batch to datacentre DC_k . After routing, tasks are appended to the corresponding local FIFO queues.

Local state. For each datacentre DC_i , the local observation $s_{i,t}^l$ captures host/VM-level utilisation, queue information, and the next task to be dispatched:

$$\mathbf{s}_{i,t}^l = (\mathbf{h}_i^{\text{load}}(t), \mathbf{h}_i^{\text{ram}}(t), \mathbf{v}_i^{\text{load}}(t), \mathbf{v}_i^{\text{type}}(t), \mathbf{v}_i^{\text{map}}(t), \mathbf{v}_i^{\text{free}}(t), Q_i(t), p_i^{\text{next}}(t)). \quad (17)$$

Here, $\mathbf{h}_i^{\text{load}}(t)$ and $\mathbf{h}_i^{\text{ram}}(t)$ are padded vectors of host CPU and RAM utilisation, $\mathbf{v}_i^{\text{load}}(t)$ is the padded VM CPU utilisation vector, $\mathbf{v}_i^{\text{type}}(t) \in \{0, 1, 2, 3\}^{|\mathcal{V}_i|}$ encodes VM types (0=off/empty, 1=small, 2=medium, 3=large), $\mathbf{v}_i^{\text{map}}(t)$ maps each VM slot to its hosting physical machine (or -1 if empty), and $\mathbf{v}_i^{\text{free}}(t)$ gives the number of free PEs per VM. Finally, $Q_i(t)$ is the local queue length and $p_i^{\text{next}}(t)$ is the PE demand of the next task to be scheduled.

Algorithm 1: HiGreen-GTrXL (Global): Batch Routing Across Datacentres

Input: Global policy $\pi_{\theta_g}^g$; global batch size B ; global queue Q^g ; datacentres $\{DC_i\}_{i=1}^N$; TimeCAP horizons k_s, k_l

Output: Routing action a_t^g and routed batch $\mathcal{B}_t$
 Receive arriving tasks $\mathcal{C}_t$; append $\mathcal{C}_t$ to Q^g ;
 // Invoke TimeCAP for green-power look-ahead

foreach datacentre DC_i **do**

Query future green power trajectory
 $\{P_i^{\text{green}}(t + \tau)\}_{\tau=1}^{k_l}$;
 Compute short-term mean:
 $\mu_i^{\text{short}} \leftarrow \frac{1}{k_s} \sum_{\tau=1}^{k_s} P_i^{\text{green}}(t + \tau) / P_i^{\text{cap}}$;
 Compute short-term trend:
 $\tau_i^{\text{short}} \leftarrow (P_i^{\text{green}}(t + k_s) - P_i^{\text{green}}(t)) / P_i^{\text{cap}}$;
 Compute long-term mean:
 $\mu_i^{\text{long}} \leftarrow \frac{1}{k_l} \sum_{\tau=1}^{k_l} P_i^{\text{green}}(t + \tau) / P_i^{\text{cap}}$;
 Compute peak timing:
 $\phi_i^{\text{peak}} \leftarrow \frac{1}{k_l} \arg \max_{\tau} P_i^{\text{green}}(t + \tau)$;

Construct global observation s_t^g (datacentre aggregates + TimeCAP features + batch features);

Select batch $\mathcal{B}_t \leftarrow \text{PeekBatch}(Q^g, B)$ (pad if $|\mathcal{B}_t| < B$);

Sample routing vector $a_t^g \sim \pi_{\theta_g}^g(\cdot | s_t^g)$, where $a_t^g \in \{1, \dots, N\}^B$;

foreach task $c_j \in \mathcal{B}_t$ **do**

Route c_j to datacentre $DC_{a_{t,j}^g}$;
 Append c_j to the corresponding local FIFO queue $Q_{a_{t,j}^g}$;

Remove $|\mathcal{B}_t|$ tasks from Q^g ;

Local action and feasibility masking. The local policy selects a VM (or defers) via

$$a_{i,t}^l \in \{0, 1, \dots, |\mathcal{V}_i|\}, \quad (18)$$

where action 0 denotes *no assignment* (keep waiting), and action $k \geq 1$ assigns the next waiting task to VM k . To prevent infeasible assignments, we employ an action mask $m_{i,t} \in \{0, 1\}^{|\mathcal{V}_i|+1}$, where $m_{i,t}(k) = 1$ iff VM k has sufficient available resources for the next task (and $m_{i,t}(0) = 1$ always holds); masked actions are either disallowed by the policy or penalised during learning.

C. Reward Design for Carbon Optimisation

Our reward design aligns with the objectives of this work: to minimise carbon emissions while trading off carbon reduction against QoS. To this end, HiGreen-GTrXL adopts a hierarchical reward structure in which the global and local agents receive different but complementary signals and cooperate to achieve system-wide carbon efficiency without sacrificing service performance. Local policies are encouraged to reduce waiting time, improve VM utilisation, and avoid infeasible

Algorithm 2: HiGreen-GTrXL (Local): VM Placement Within Each Datacentre

Input: Local policy $\pi_{\theta_l}^l$ (shared or per-DC); local FIFO queues $\{Q_i\}_{i=1}^N$; VM sets $\{\mathcal{V}_i\}_{i=1}^N$

Output: Local placement actions $\{a_{i,t}^l\}_{i=1}^N$

for $i = 1$ **to** N **do**

if Q_i *is not empty* **then**

Construct local observation $s_{i,t}^l$ (host/VM utilisation, VM types, free PEs, queue info, next-task demand);

Compute feasibility mask $m_{i,t} \in \{0, 1\}^{|\mathcal{V}_i|+1}$ for the next task in Q_i ;

Sample action $a_{i,t}^l \sim \pi_{\theta_l}^l(\cdot | s_{i,t}^l, m_{i,t})$, where $a_{i,t}^l \in \{0, 1, \dots, |\mathcal{V}_i|\}$;

if $a_{i,t}^l \neq 0$ **then**

Pop the head task from Q_i ; assign it to VM $a_{i,t}^l$ in DC_i ;

actions, while the global policy additionally penalises carbon emissions and renewable curtailment.

Local reward. For each datacentre DC_i , we define the local reward at step t as

$$r_{i,t}^l = r_{i,t}^{\text{wait}} + r_{i,t}^{\text{util}} + r_{i,t}^{\text{queue}} + r_{i,t}^{\text{inv}}, \quad (19)$$

where the components follow the simulator implementation. First, the waiting-time penalty is computed from the set of waiting times of tasks that finish within the current step:

$$r_{i,t}^{\text{wait}} = -\omega_{\text{wait}} \log(1 + \bar{T}_{i,t}^{\text{wait}}). \quad (20)$$

Second, we penalise low utilisation and imbalance using active VM CPU utilisation in DC_i . Let $\bar{U}_{i,t}^{\text{vm}}$ be the mean VM utilisation and $\sigma_{i,t}^{2,\text{vm}}$ be the corresponding variance; then

$$r_{i,t}^{\text{util}} = -\omega_{\text{util}} \left(\sqrt{\sigma_{i,t}^{2,\text{vm}}} + |\bar{U}_{i,t}^{\text{vm}} - U^*| \right), \quad (21)$$

where U^* is a target utilisation (set to 0.75 in our multi-datacentre setting). Third, we penalise queue backlog by normalising the local queue length $Q_i(t)$ by the cumulative number of tasks routed to DC_i , denoted by $N_i^{\text{rec}}(t)$:

$$r_{i,t}^{\text{queue}} = -\omega_{\text{queue}} \frac{Q_i(t)}{\max\{1, N_i^{\text{rec}}(t)\}}. \quad (22)$$

Finally, infeasible actions are penalised with a flat term:

$$r_{i,t}^{\text{inv}} = -\omega_{\text{inv}} \cdot \mathbb{I}_{i,t}^{\text{inv}}, \quad (23)$$

where $\mathbb{I}_{i,t}^{\text{inv}} = 1$ if the local action at step t is invalid (e.g., selecting a non-existent VM or attempting placement with an empty queue), and 0 otherwise.

Global reward for carbon optimisation. The global reward combines average local performance with system-wide carbon and renewable-waste penalties:

$$r_t^g = \alpha \bar{r}_t^l - \beta \hat{C}_t - \kappa R_w(t), \quad (24)$$

where $\bar{r}_t^l = \frac{1}{N} \sum_{i=1}^N r_{i,t}^l$. The carbon penalty uses a running-max normalisation to stabilise learning:

$$\hat{C}_t = \min \left(\frac{\sum_{i=1}^N \Delta C_i(t)}{M_C + \epsilon}, c_{\max} \right), \quad (25)$$

where M_C is the running maximum of $\sum_i \Delta C_i(t)$ observed so far, ϵ avoids division by zero, and $c_{\max}$ caps extreme ratios. The renewable-waste ratio is

$$R_w(t) = \frac{\sum_{i=1}^N \Delta E_i^{\text{waste}}(t)}{\sum_{i=1}^N (\Delta E_i^{\text{waste}}(t) + \Delta E_i^{\text{green}}(t))} \in [0, 1]. \quad (26)$$

This design encourages the global policy to route workloads towards datacentres with cleaner electricity and better renewable availability, while local policies maintain QoS and feasibility.

D. Policy Network Architecture and Parameter Sharing

We parameterise both the global routing policy π^g and the local scheduling policies $\{\pi_i^l\}_{i=1}^N$ with neural networks that map observations to action distributions. To capture temporal dependencies induced by non-stationary workloads and time-varying renewable supply, we adopt Gated Transformer-XL (GTrXL) as the policy backbone. Unless otherwise stated, the global and local policies use the same backbone configuration while maintaining separate heads for their respective action spaces.

At each step, the global network receives the aggregated observation s_t^g , which includes per-datacentre metrics, TimeCAP look-ahead features, and batch-level task descriptors. The local network receives $s_{i,t}^l$, comprising host/VM utilisation vectors, VM-type indicators, resource availability, and queue-related features. Continuous features are normalised to $[0, 1]$ where appropriate, and categorical features (e.g., VM types) are represented via learned embedding vectors. The resulting feature tokens are then fed into the GTrXL layers.

GTrXL employs segment-level recurrence to maintain a fixed-length memory of past hidden states, enabling the policy to leverage historical context beyond a single step. In our implementation, the backbone is configured with L Transformer layers, hidden dimension d , H attention heads, feed-forward width d_{ff} , and memory length m , as specified in the experimental configuration. This design allows the agent to reason about temporal patterns such as renewable trends and queue build-up without explicitly hand-crafting history features. The combination of GTrXL's temporal modelling with TimeCAP's explicit look-ahead features provides complementary sources of temporal information: GTrXL captures implicit patterns from observation history, while TimeCAP supplies explicit forecasts of future renewable availability.

The global policy head outputs a routing distribution over $\{1, \dots, N\}^B$ for batch routing, while each local policy head outputs a distribution over $\{0, 1, \dots, |\mathcal{V}_i|\}$, where action 0 denotes deferring placement. To ensure feasibility, we apply action masking to the local head by zeroing out logits of invalid VM choices (e.g., VMs with insufficient free PEs for the next task) and renormalising the remaining probabilities.

To improve sample efficiency and reduce the number of trainable parameters, we adopt a CTDE paradigm with parameter sharing among local schedulers. All local agents share the same network parameters θ_l while operating on different local observations. Training is centralised in the sense that trajectories from both the global policy and all local policies are collected and optimised jointly via PPO. At runtime, execution remains fully decentralised: each agent acts solely based on its own observation without requiring communication with other agents, enabling scalable deployment across geographically distributed datacentres.

V. PERFORMANCE EVALUATION

A. Experiment Setup

This work is tested by utilising the CloudSim Plus framework [34], a discrete-event simulator for cloud datacentre environments. CloudSim Plus provides the core simulation infrastructure in our implementation, including cloud task execution, VM provisioning, host resource accounting, and power modelling. We model geo-distributed datacentres with heterogeneous compute infrastructures. Each datacentre is configured with a single server profile (host type) and a pre-provisioned VM fleet with three VM sizes (Small/Medium/Large). Host power consumption is modelled as a function of CPU utilisation using empirical server power curves derived from the SPECpower_ss2008 benchmark suite [5], and the adopted server profiles are summarised in Table II. These curves provide realistic idle-to-peak power characteristics for different server platforms, and are used by CloudSim Plus to compute instantaneous IT power based on utilisation. For the DRL framework and environment interface, the learning pipeline Ray RLlib [35] is applied, and the Java-based simulator is exposed by Py4J and PettingZoo [36] as a multi-agent environment. The hierarchical setting is represented as a global routing agent and multiple local scheduling agents (one per datacentre), enabling centralised training and decentralised execution within RLlib. Each datacenter is associated with a renewable-power trace (wind or solar), collected by two public datasets for wind power input and solar power input respectively [37] [38].

B. Workload Dataset

We evaluate HiGreen-GTrXL under three synthetic workload intensities (5k, 15k, and 100k) generated by our workload generator. The generator outputs a CSV trace of cloudlets, each described by (`cloudlet_id`, `arrival_time`, `length`, `pes_required`, `file_size`, `output_size`).

Arrivals. Unless otherwise stated, arrivals follow a Poisson process with rate λ jobs/s (exponential inter-arrival times) over a duration T . We also support uniform and bursty patterns.

Job characteristics. Cloudlet length (MI) is sampled from a configurable distribution within $[L_{\min}, L_{\max}]$, and the PE demand is an integer in $[1, 8]$ with a distribution biased towards smaller jobs (weighted by default). Input size is proportional

to length (approximately 1 KB per 1000 MI), and output size is half of the input.

Reproducibility. All traces are generated with a fixed random seed. The 5k, 15k, and 100k workloads differ only in intensity (arrival rate/job count), while keeping job-size and PE distributions unchanged.

C. Baselines

We compare against five representative baselines. **Round-Robin** is a simple heuristic that routes incoming tasks to datacenters in a cyclic order, serving as a lightweight load-balancing reference. **GA** (Genetic Algorithm) and **PSO** (Particle Swarm Optimization) are population-based metaheuristics that search for routing/scheduling decisions by iteratively improving candidate solutions via evolutionary operators (GA) or swarm updates (PSO), providing strong optimization-oriented baselines. **PPO without prediction** is a reinforcement-learning baseline trained with Proximal Policy Optimization using only current observable system states (no renewable prediction features), capturing the benefit of RL without foresight. **PPO-MLP with prediction** uses PPO with an MLP policy under an oracle-style global observation (“god-eye”), enabling access to more complete system information and serving as an upper-reference RL baseline under enhanced observability.

D. Results

We evaluate our hierarchical multi-agent scheduler through training dynamics analysis and comparative benchmarks. Fig. 3 presents the reward evolution throughout training. The episode reward improves from approximately $-46,000$ to $-38,000$ over 100 episodes, with the smoothed curve showing a clear upward trajectory. The global agent reward exhibits similar improvement from around $-5,400$ to $-4,600$, indicating that the routing policy progressively learns to distribute workloads more effectively across datacenters. The average local agent reward also improves from $-4,200$ to $-3,400$, confirming that VM-level placement decisions co-evolve with the global routing strategy rather than lagging behind.

Fig. 4 reveals the learning dynamics of individual local agents. Despite operating under heterogeneous conditions with varying datacenter capacities and green energy availability, all ten agents exhibit upward reward trends. The agents start from different initial performance levels ranging from $-5,000$ to $-3,200$, reflecting the inherent differences across datacenters. By the end of training, rewards converge to a narrower band between $-3,200$ and $-3,600$, demonstrating that the hierarchical architecture successfully allows each local policy to adapt to its specific environment.

Fig. 5 illustrates the carbon-related metrics. The total carbon emission per episode decreases from 1.707 kg CO_2 to 1.701 kg CO_2 , indicating that the learned policies reduce reliance on carbon-intensive energy. The carbon intensity stabilises around 0.2624 kg/kWh with limited variance after the initial training phase, suggesting that carbon reduction is achieved

through consistent scheduling behaviour rather than erratic energy shifting.

Fig. 6 compares our hierarchical approach against baselines including round-robin, metaheuristic methods (PSO, PSO_GA), and alternative reinforcement learning configurations across three workload scales. At smaller scales (5k), all methods achieve comparable performance. As the workload increases to 15k and 100k cloudlets, the advantages of our approach become more evident. At the 100k scale, our method achieves the lowest carbon intensity among learning-based approaches at approximately 0.4221 kg/kWh while maintaining competitive energy consumption. These results demonstrate that the hierarchical decomposition enables effective scaling, where the global agent routes tasks to datacenters with higher renewable availability while local agents optimise placement to maximise green energy utilisation.

VI. CONCLUSION

This paper studied bilateral green-energy alignment for geo-distributed datacentres and argued that reactive strategies are insufficient under volatile renewable supply and limited storage. To proactively match computing demand with forthcoming renewable availability, we introduced HiGreen-GTrXL, a hierarchical DRL framework that coordinates cross-datacentre routing and intra-datacentre VM scheduling, while injecting multi-horizon green-power look-ahead signals into the global state via a forecasting module. By jointly optimising carbon-aware objectives and service-quality considerations in a unified sequential decision process, our approach provides an effective mechanism for shifting workload toward future renewable-rich locations and reducing curtailment. These findings highlight the value of integrating lightweight renewable forecasting with hierarchical control for sustainable cloud scheduling, and motivate further work on robustness to forecast uncertainty and deployment-oriented constraints.

REFERENCES

- [1] W. Lin, J. Lin, Z. Peng, H. Huang, W. Lin, and K. Li, “A systematic review of green-aware management techniques for sustainable data center,” *Sustainable Computing: Informatics and Systems*, vol. 42, p. 100989, Apr. 2024. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2210537924000349>
- [2] K. Zhang, P. Wang, N. Gu, and T. D. Nguyen, “GreenDRL: managing green datacenters using deep reinforcement learning,” in *Proceedings of the 13th Symposium on Cloud Computing*. San Francisco California: ACM, Nov. 2022, pp. 445–460. [Online]. Available: <https://dl.acm.org/doi/10.1145/3542929.3563501>
- [3] V. Venkataswamy, J. Grigsby, A. Grimshaw, and Y. Qi, “RARE: Renewable Energy Aware Resource Management in Datacenters,” in *Job Scheduling Strategies for Parallel Processing: 25th International Workshop, JSPP 2022, Virtual Event, June 3, 2022, Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2022, pp. 108–130. [Online]. Available: https://doi.org/10.1007/978-3-031-22698-4_6
- [4] L. Xue, J. Wang, H. Li, W. Yong, and Y. Zhang, “Online energy conservation scheduling for geo-distributed data centers with hybrid data-driven and knowledge-driven approach,” *Energy*, vol. 322, p. 135714, May 2025. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0360544225013568>

TABLE II
SERVER PROFILES USED IN OUR SIMULATOR (DERIVED FROM SPECPOWER_Ssj2008)

Server Name	Processor	Cores	MIPS	RAM	Power (W)	
					Idle	Active
Acer Altos R520	Intel Xeon E5450	8	400k	16 GB	155.0	269.0
Acer AR360 F2	Intel Xeon E5-2660	16	800k	32 GB	69.4	315.0
ASUSTeK RS720-E9/RS8	Intel Xeon Platinum 8180 (2×28)	56	2800k	128 GB	48.2	385.0
ASUSTeK RS500A-E10-PS4	AMD EPYC 7742	64	3200k	256 GB	51.4	214.0
ASUSTeK RS700A-E9-RS4V2	AMD EPYC 7742 (2×64)	128	6400k	512 GB	106.0	430.0

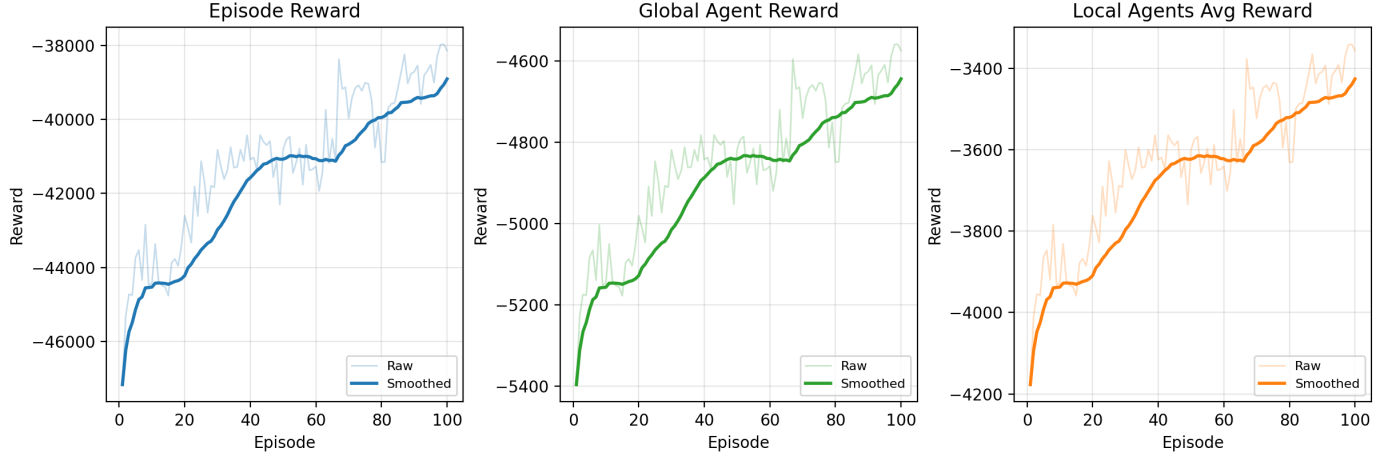


Fig. 3. Training reward curves showing episode reward (left), global agent reward (middle), and average local agent reward (right). All metrics exhibit consistent improvement over 100 training episodes.

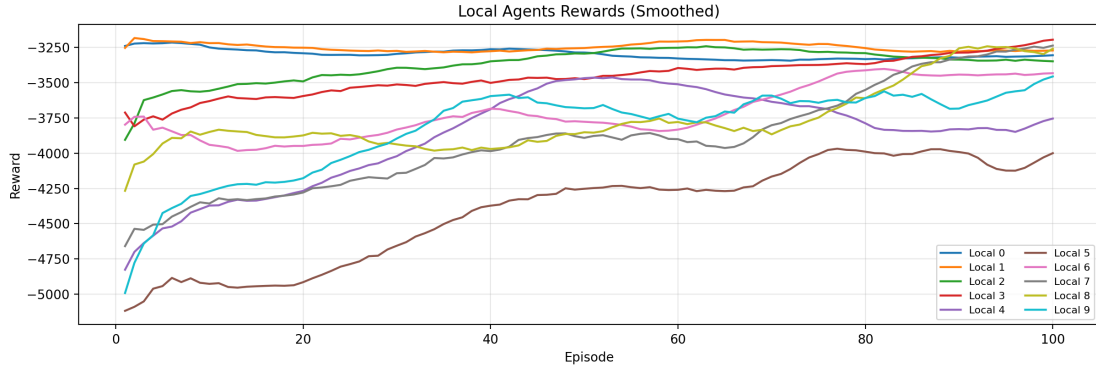


Fig. 4. Individual local agent reward trajectories across ten datacenters. Despite heterogeneous conditions, all agents demonstrate upward learning trends.

- [5] K.-D. Lange, M. G. Tricker, J. A. Arnold, H. Block, and S. Sharma, "SPECpower_ssj2008: driving server energy efficiency," in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 253–254, event-place: Boston, Massachusetts, USA. [Online]. Available: <https://doi.org/10.1145/2188286.2188329>
- [6] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: state-of-the-art and research challenges," *Journal of Internet Services and Applications*, vol. 1, pp. 7–18, Apr. 2010.
- [7] G. L. Stavrinides and H. D. Karatza, "An energy-efficient, QoS-aware and cost-effective scheduling approach for real-time workflow applications in cloud computing systems utilizing DVFS and approximate computations," *Future Generation Computer Systems*, vol. 96, pp. 216–226, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X18327353>
- [8] M. Haddad, J.-M. Nicod, M.-C. Marion-Péra, and C. Varnier, "Stand-alone renewable power system scheduling for a green data center using integer linear programming," *Journal of Scheduling*, vol. 24, Oct. 2021.
- [9] A. A. Bhattacharya, D. Culler, E. Friedman, A. Ghodsi, S. Shenker, and I. Stoica, "Hierarchical scheduling for diverse datacenter workloads," in *Proceedings of the 4th annual Symposium on Cloud Computing*. Santa Clara California: ACM, Oct. 2013, pp. 1–15. [Online]. Available: <https://dl.acm.org/doi/10.1145/2523616.2523637>
- [10] A. Jayanetti and R. Buyya, "J-OPT: A Joint Host and Network Optimization Algorithm for Energy-Efficient Workflow Scheduling in Cloud Data Centers," in *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*. Auckland New Zealand: ACM, Dec. 2019, pp. 199–208. [Online]. Available: <https://dl.acm.org/doi/10.1145/3344341.3368822>

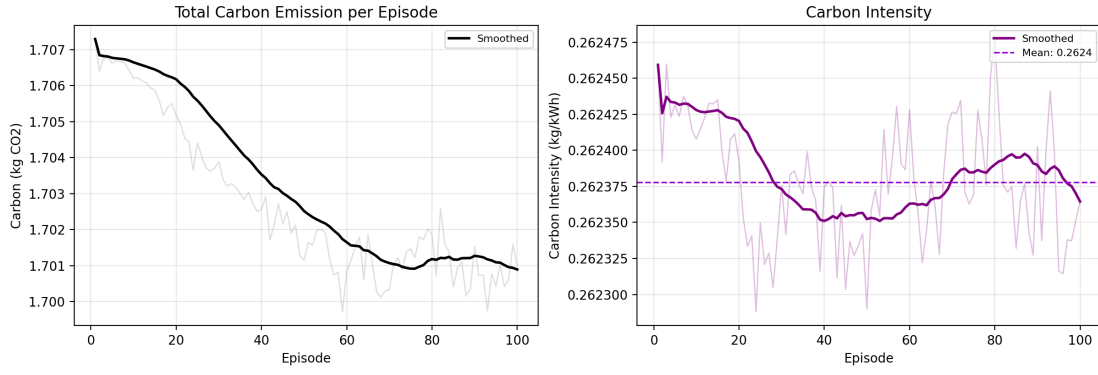


Fig. 5. Carbon metrics during training: total carbon emission per episode (left) and carbon intensity (right). The dashed line indicates the mean carbon intensity of 0.2624 kg/kWh.

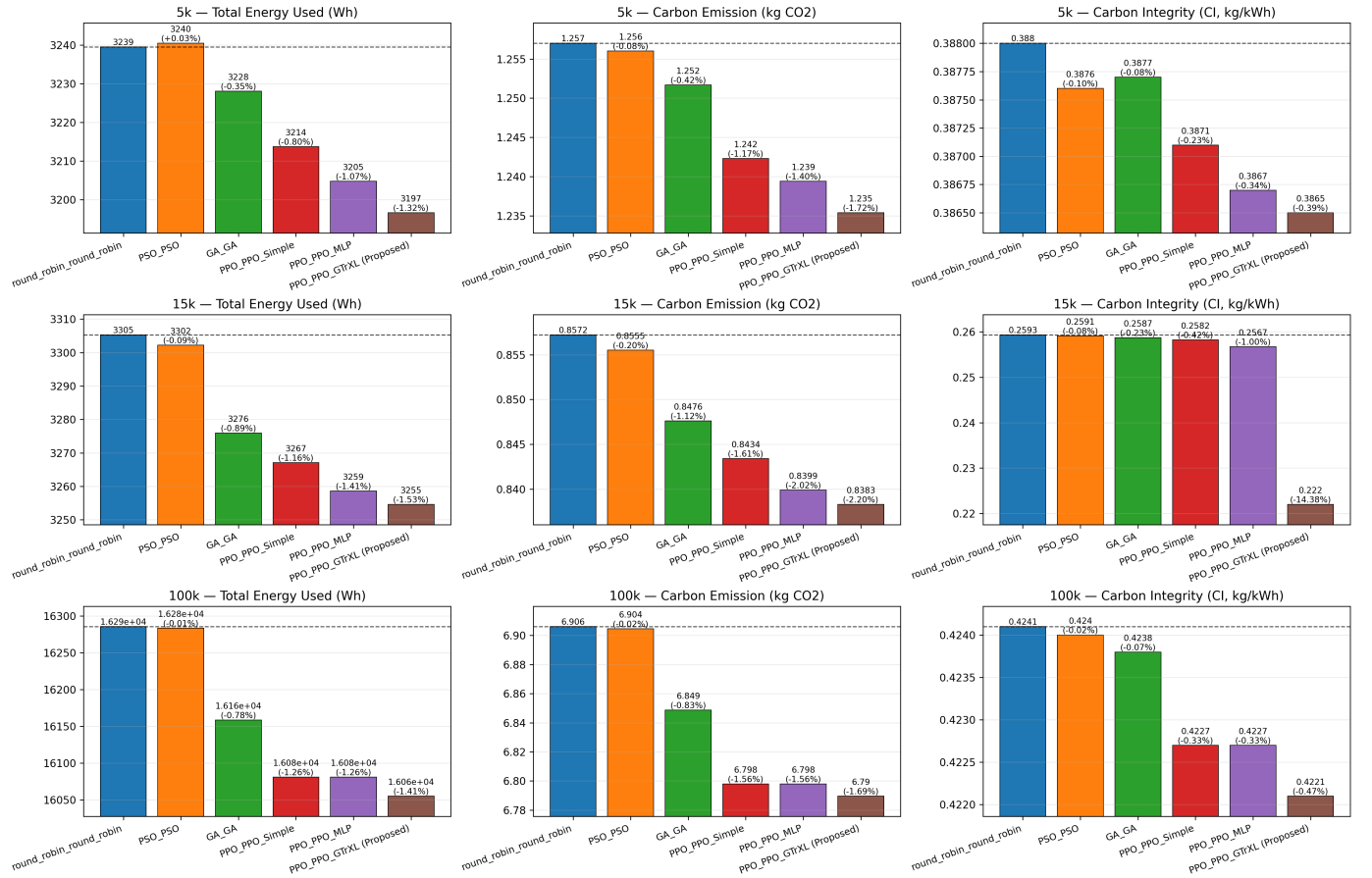


Fig. 6. Performance comparison across different workload scales (5k, 15k, 100k cloudlets) in terms of total energy consumption, carbon emission, and carbon intensity.

- [11] R. Wang, Y. Lu, K. Zhu, J. Hao, P. Wang, and Y. Cao, "An Optimal Task Placement Strategy in Geo-Distributed Data Centers Involving Renewable Energy," *IEEE Access*, vol. 6, pp. 61948–61958, 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8493571/>
- [12] A. Agarwal, J. Sun, S. Noghabi, S. Iyengar, A. Badam, R. Chandra, S. Seshan, and S. Kalyanaraman, "Redesigning Data Centers for Renewable Energy," in *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks*. Virtual Event United Kingdom: ACM, Nov. 2021, pp. 45–52. [Online]. Available: <https://dl.acm.org/doi/10.1145/3484266.3487394>
- [13] M. Haddad, J. M. Nicod, C. Varnier, and M.-C. Peera, "Mixed Integer Linear Programming Approach to Optimize the Hybrid Renewable Energy System Management for supplying a Stand-Alone Data Center," in *2019 Tenth International Green and Sustainable Computing Conference (IGSC)*. Alexandria, VA, USA: IEEE, Oct. 2019, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/document/8957199/>
- [14] X. Ren, J. Wang, X. Hu, Z. Sun, Q. Zhao, D. Chong, K. Xue, and J. Yan, "A novel demand response-based distributed multi-energy system optimal operation framework for data centers," *Energy and Buildings*, vol. 305, p. 113886, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378778824000021>
- [15] F. A. Omara and M. M. Arafa, "Genetic algorithms for task

- scheduling problem,” *Journal of Parallel and Distributed Computing*, vol. 70, no. 1, pp. 13–22, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731509001804>
- [16] A. Pradhan, S. K. Bisoy, and A. Das, “A survey on PSO based meta-heuristic scheduling mechanism in cloud computing environment,” *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 8, Part A, pp. 4888–4901, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157821000033>
- [17] A. F. Minai, “Metaheuristics Paradigms for Renewable Energy Systems: Advances in Optimization Algorithms,” Jan. 2021, pp. 35–61.
- [18] S. Iturriaga, S. Nesmachnow, A. Tchernykh, and B. Dorronsoro, “Multiobjective Workflow Scheduling in a Federation of Heterogeneous Green-Powered Data Centers,” in *2016 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, 2016, pp. 596–599.
- [19] P. Hosseinioun, M. Kheirabadi, S. R. K. Tabbakh, and R. Ghaemi, “A new energy-aware tasks scheduling approach in fog computing using hybrid meta-heuristic algorithm,” *Journal of Parallel and Distributed Computing*, vol. 143, pp. 88–96, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S074373152030023X>
- [20] S. M. Mirmohseni, C. Tang, and A. Javadpour, “FPSO-GA: A Fuzzy Metaheuristic Load Balancing Algorithm to Reduce Energy Consumption in Cloud Networks,” *Wireless Personal Communications*, vol. 127, no. 4, pp. 2799–2821, Dec. 2022. [Online]. Available: <https://link.springer.com/10.1007/s11277-022-09897-3>
- [21] J. Kakkottakath Valappil Thekkepurayil, D. P. Suseelan, and P. M. Keerikkattil, “An effective meta-heuristic based multi-objective hybrid optimization method for workflow scheduling in cloud computing environment,” *Cluster Computing*, vol. 24, no. 3, pp. 2367–2384, Sep. 2021. [Online]. Available: <https://link.springer.com/10.1007/s10586-021-03269-5>
- [22] M. Abdel-Basset, D. El-Shahat, M. Elhoseny, and H. Song, “Energy-Aware Metaheuristic Algorithm for Industrial-Internet-of-Things Task Scheduling Problems in Fog Computing Applications,” *IEEE Internet of Things Journal*, vol. 8, no. 16, pp. 12 638–12 649, Aug. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9153148/>
- [23] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource Management with Deep Reinforcement Learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, ser. HotNets ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 50–56, event-place: Atlanta, GA, USA. [Online]. Available: <https://doi.org/10.1145/3005745.3005750>
- [24] M. Xu, A. N. Toosi, and R. Buyya, “A Self-Adaptive Approach for Managing Applications and Harnessing Renewable Energy for Sustainable Cloud Computing,” *IEEE Transactions on Sustainable Computing*, vol. 6, no. 4, pp. 544–558, Oct. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9162490/>
- [25] J. Gao, H. Wang, and H. Shen, “Smartly Handling Renewable Energy Instability in Supporting A Cloud Datacenter,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. New Orleans, LA, USA: IEEE, May 2020, pp. 769–778. [Online]. Available: <https://ieeexplore.ieee.org/document/9139872/>
- [26] Q. Yu, M. Hu, Z. Yang, and X. Zhou, “Carbon-Efficient Data Center Scheduling under Uncertainty: A Conformal Prediction Approach,” Mar. 2025, pp. 232–236.
- [27] C. Xu, K. Wang, P. Li, R. Xia, S. Guo, and M. Guo, “Renewable energy-aware big data analytics in geo-distributed data centers with reinforcement learning,” *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 1, pp. 205–215, 2018, publisher: IEEE.
- [28] J. Zhao, M. A. Rodríguez, and R. Buyya, “A Deep Reinforcement Learning Approach to Resource Management in Hybrid Clouds Harnessing Renewable Energy and Task Scheduling,” in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*, 2021, pp. 240–249.
- [29] C. Gu, C. Liu, J. Zhang, H. Huang, and X. Jia, “Green scheduling for cloud data centers using renewable resources,” in *2015 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. Hong Kong, Hong Kong: IEEE, Apr. 2015, pp. 354–359. [Online]. Available: <http://ieeexplore.ieee.org/document/7179410/>
- [30] S. Zhang, M. Xu, W. Y. B. Lim, and D. T. Niyato, “Sustainable AIGC Workload Scheduling of Geo-Distributed Data Centers: A Multi-Agent Reinforcement Learning Approach,” *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, pp. 3500–3505, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258180402>
- [31] A. Jayanetti, S. Halgamuge, and R. Buyya, “Multi-Agent Deep Reinforcement Learning Framework for Renewable Energy-Aware Workflow Scheduling on Distributed Cloud Data Centers,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 604–615, Apr. 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10417832/>
- [32] D. Niewenhuis, S. Talluri, A. Iosup, and T. De Matteis, “FootPrinter: Quantifying Data Center Carbon Footprint,” in *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*. London United Kingdom: ACM, May 2024, pp. 189–195. [Online]. Available: <https://dl.acm.org/doi/10.1145/3629527.3651419>
- [33] C. Ren, Y. Lu, T. Huang, H. Zheng, H. Zhu, Y. Li, H. Li, and L. Liu, “TimeCAP: A Channel-Aware Pre-Training Framework for Multivariate Time Series Forecasting,” Dec. 2025. [Online]. Available: https://ore.exeter.ac.uk/articles/conference_contribution/TimeCAP_A_Channel-Aware_Pre-Training_Framework_for_Multivariate_Time_Series_Forecasting/30773819
- [34] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. Inacio, and M. M. Freire, “CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness,” in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*. Lisbon: IEEE, May 2017, pp. 400–406. [Online]. Available: <https://ieeexplore.ieee.org/document/7987304/>
- [35] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica, “RLlib: Abstractions for Distributed Reinforcement Learning,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, Jul. 2018, pp. 3053–3062. [Online]. Available: <https://proceedings.mlr.press/v80/liang18b.html>
- [36] J. K. Terry, B. Black, A. Hari, L. S. Santos, C. Dieffendahl, N. L. Williams, Y. Lokesh, C. Horsch, and P. Ravi, “PettingZoo: Gym for Multi-Agent Reinforcement Learning,” *CoRR*, vol. abs/2009.14471, 2020, arXiv: 2009.14471. [Online]. Available: <https://arxiv.org/abs/2009.14471>
- [37] J. Zhou, “SDWPF: A Dataset for Spatial Dynamic Wind Power Forecasting over a Large Turbine Array,” Dec. 2023. [Online]. Available: https://figshare.com/articles/dataset/SDWPF_dataset/24798654
- [38] pythonafroz, “Solar Power Generation Forecast,” 2021. [Online]. Available: <https://www.kaggle.com/code/pythonafroz/solar-power-generation-forecast/notebook>